SHoEbox: structural optimization stage 1

13 November 2026 | Baseline: `8030a1e749d9e67563ccde9b7101851bc40d2a26`

Implemented the first item in the review's proposed implementation order: remove overwritten adapter work and compact completion learning records. The default reference path uses the update. This is primarily a memory improvement; short-run training and inference speed changes are modest.

Implementation

- Build the structural descriptor and eight symbol features once. Evaluate the semantic adapter only in the final constituent encoder, preserving arithmetic order and intervention accounting.

- Remove unused legacy semantic edges, payload-addition derivatives and duplicated symbol arrays from completion input credits. Preserve the saved eligibility gate, semantic handle, feedback Jacobians, routing derivatives and pending gradients.

- Store bottom temporal records only for A1/B1, separately from the shared neural archive. Reclaim and reset both stores together.

- Preserve checkpoint v39, composition ABI 10, the explicit ABI 9 reader and neural archive ABI 1 through dedicated wire-layout conversion. Historical kernels retain their original layouts.

Memory reduction

Record or workload	Before	After	Reduction
Completion input credit	5,488 B	1,552 B	71.7%
Upper retained neural record	11,424 B	5,400 B	52.7%
Bottom record plus temporal payload	11,424 B	7,488 B	34.5%
Fixed composition runtime	32.9 MiB	28.6 MiB	13.1%
Archive after 2,048 total targets	94.8 MiB	52.4 MiB	44.7%
Archive after 5,120 total targets	188.1 MiB	107.1 MiB	43.0%

These are record payload bytes, excluding map/allocator overhead and other model buffers. The longer run retains 17,263 neural records and 9,160 bottom temporal records. `model.completion_storage()` exposes the sizes and live totals. Checkpoint wire records remain expanded for compatibility.

Matched throughput

Workload	Before, B/s	After, B/s	Change
1M repeated text, 1,024 targets	176.5	182.4	+3.3%
1M natural text, 1,024 targets	258.1	257.2	-0.3%
1M frozen inference, 4,096 targets	2,980.3	3,095.4	+3.9%
5M repeated text, 1,024 targets	126.3	128.9	+2.1%
1M repeated text, 4,096 targets	71.5	81.0	+13.3%

One pinned CPU on a Xeon Platinum 8573C, serial execution, width 64, all reference features enabled. Each continuation starts from the same actual baseline-runtime checkpoint after 1,024 warmup targets. Timings include full logit export and exclude compilation, checkpoint I/O and hashing. Implementation order alternates. The first three rows use five pairs; 5M uses three pairs; the longer training row is one pair. The regression and sanitizer workloads finished before these timings began. Small percentage changes overlap observed timing variation.

Verification and compatibility

- 231 existing regression tests passed. Three new storage/adapter tests passed in a fresh process, and the same three passed with 64 experts per region (5M).

- ASan/UBSan passed training, evaluation, drain/reset, checkpoint round trips and a growing live semantic graph beyond 16,384 records.

- All 38 measured continuations match bit-for-bit for logits, NLL, fitted parameter groups, routing codes, recurrent/base state, content-expert eligibility, pattern state and paired episodic state. No lost-credit integrity checks failed.

- Actual old-to-new-to-old checkpoint continuation passes, including an evaluation interval followed by resumed training. Chunking, delayed credit and bottom-state lifetime are covered by the regressions.

The broad test process had imported an earlier version of the new test fixture's setup before that fixture was finalized. Its sole setup error was a removed helper name; all 231 existing tests passed. The finalized three tests were rerun successfully in a fresh process. Raw logs and the validation summary record this explicitly.

Unused historical eligibility fields are written as zeros. Whole serialized fingerprints therefore differ from the old runtime; these bytes are not read by completion learning. New-runtime save/load fingerprints remain exact, and the old runtime successfully resumes newly exported checkpoints.

Reproduction

Use `src/compare_optimized_runtime.py` against an unchanged checkout of the baseline commit. The recorded JSON files under `results/storage_*.json` contain source hashes, workload hashes, timings and parity checks. `src/check_storage_compatibility.py` verifies both-direction checkpoint continuation; add `--bridge-mode eval` for evaluation followed by training. The package README includes complete commands.